

Tasks › Administer-Cluster › Namespaces-Walkthrough

Namespaces Walkthrough | Kubernetes

Namespaces WalkthroughKubernetes namespaces

help different projects, teams, or customers to share a Kubernetes cluster. It does this by providing the following: A scope for Names. A mechanism to attach authorization and policy to a subsection of the cluster. Use of multiple namespaces is optional. This example demonstrates how to use Kubernetes namespaces to subdivide your cluster. Before you begin You need to have a Kubernetes cluster, and the kubectl command-line tool must

be configured to communicate with your cluster. It is recommended to run this tutorial on a cluster with at least two nodes that are not acting as control plane hosts. If you do not already have a cluster, you can create one by using

minikube

or you can use one of these Kubernetes playgrounds: iximiuz Labs Killercoda KodeKloud To check the version, enter `kubectl version`. Prerequisites This example assumes the following: You have an existing Kubernetes cluster. You have a basic understanding of Kubernetes Pods, Services, and Deployments. Understand the default namespace By default, a Kubernetes cluster will instantiate a default namespace when provisioning the cluster to hold the default set of Pods,

Services, and Deployments used by the cluster. Assuming you have a fresh cluster, you can inspect the available namespaces by doing the following: `kubectl get namespaces`

```
NAME STATUS AGE
```

```
default Active 13m
```

Create new namespaces For this exercise, we will create two additional Kubernetes namespaces to hold our content. Let's imagine a scenario where an organization is using a shared Kubernetes cluster for development and production use cases. The development team would like to maintain a space in the cluster where they can get a view on the list of Pods, Services, and Deployments

they use to build and run their application. In this space, Kubernetes resources come and go, and the restrictions on who can or cannot modify resources

are relaxed to enable agile development. The operations team would like to maintain a space in the cluster where they can enforce strict procedures on who can or cannot manipulate the set of

Pods, Services, and Deployments that run the production site. One pattern this organization could follow is to partition the Kubernetes cluster into two namespaces: development and production. Let's create two new namespaces to hold our work. Use the file `namespace-dev.yaml` which describes a development

```
namespace: admin/namespace-dev.yaml
```

```
apiVersion: v1
```

```
kind: Namespace
```

```
metadata:
```

```
  name: development
```

```
  labels:
```

```
    name: development
```

Create the development namespace using kubectl. `kubectl create -f`

```
https://k8s.io/examples/admin/namespace-dev.yaml
```

Save the following contents into file `namespace-prod.yaml` which describes a production

```
namespace: admin/namespace-prod.yaml
```

```
apiVersion: v1
```

```
kind: Namespace
```

```
metadata:
```

```
  name: production
```

```
  labels:
```

```
    name: production
```

And then let's create the production namespace using kubectl. `kubectl create -f`

```
https://k8s.io/examples/admin/namespace-prod.yaml
```

To be sure things are right, let's list all of the namespaces in our cluster. `kubectl get namespaces --show-labels`

NAME STATUS AGE LABELS

default Active 32m <none>

development Active 29s name=development

production Active 23s name=production

Create pods in each namespaceA Kubernetes namespace provides the scope for Pods, Services, and Deployments in the cluster.Users interacting with one namespace do not see the content in another namespace.To demonstrate this, let's spin up a simple Deployment and Pods in the development namespace.We first check what is the current context:kubectl config view

apiVersion: v1

clusters:

- cluster:

certificate-authority-data: REDACTED

server: https://130.211.122.180

name: lithe-cocoa-92103_kubernetes

contexts:

- context:

cluster: lithe-cocoa-92103_kubernetes

user: lithe-cocoa-92103_kubernetes

name: lithe-cocoa-92103_kubernetes

current-context: lithe-cocoa-92103_kubernetes

kind: Config

preferences: {}

users:

- name: lithe-cocoa-92103_kubernetes

user:

client-certificate-data: REDACTED

client-key-data: REDACTED

token: 65rZW78y8HbwXXtSXuUw9DbP4FLjHi4b

- name: lithe-cocoa-92103_kubernetes-basic-auth

user:

password: h5M0FtUUIfIBSdI7

username: admin

kubectl config current-context

lithe-cocoa-92103_kubernetes

The next step is to define a context for the kubectl client to work in each namespace. The value of "cluster" and "user" fields are copied from the current context.kubectl config set-context dev --namespace=development \

--cluster=lithe-cocoa-92103_kubernetes \

--user=lithe-cocoa-92103_kubernetes

kubectl config set-context prod --namespace=production \

--cluster=lithe-cocoa-92103_kubernetes \

--user=lithe-cocoa-92103_kubernetes

By default, the above commands add two contexts that are saved into file

.kube/config. You can now view the contexts and alternate against the two

new request contexts depending on which namespace you wish to work against.To view the new

contexts:kubectl config view

apiVersion: v1

clusters:

- cluster:

certificate-authority-data: REDACTED

server: https://130.211.122.180

name: lithe-cocoa-92103_kubernetes

contexts:

- context:

```
cluster: lithe-cocoa-92103_kubernetes
user: lithe-cocoa-92103_kubernetes
name: lithe-cocoa-92103_kubernetes
- context:
cluster: lithe-cocoa-92103_kubernetes
namespace: development
user: lithe-cocoa-92103_kubernetes
name: dev
- context:
cluster: lithe-cocoa-92103_kubernetes
namespace: production
user: lithe-cocoa-92103_kubernetes
name: prod
current-context: lithe-cocoa-92103_kubernetes
kind: Config
preferences: {}
users:
- name: lithe-cocoa-92103_kubernetes
user:
client-certificate-data: REDACTED
client-key-data: REDACTED
token: 65rZW78y8HbwXXtSXuUw9DbP4FLjHi4b
- name: lithe-cocoa-92103_kubernetes-basic-auth
user:
password: h5M0FtUUIfIBSdI7
username: admin
```

Let's switch to operate in the development namespace.`kubectl config use-context dev`
You can verify your current context by doing the following:`kubectl config current-context dev`

At this point, all requests we make to the Kubernetes cluster from the command line are scoped to the development namespace. Let's create some contents.`admin/snowflake-deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
labels:
app: snowflake
name: snowflake
spec:
replicas: 2
selector:
matchLabels:
app: snowflake
template:
metadata:
labels:
app: snowflake
spec:
containers:
- image: registry.k8s.io/serve_hostname
imagePullPolicy: Always
name: snowflake
```

Apply the manifest to create a Deployment`kubectl apply -f https://k8s.io/examples/admin/snowflake-deployment.yaml`

We have created a deployment whose replica size is 2 that is running the pod called snowflake with a basic container that serves the hostname.`kubectl get deployment`

NAME READY UP-TO-DATE AVAILABLE AGE

snowflake 2/2 2 2 2m

kubect! get pods -l app=snowflake

NAME READY STATUS RESTARTS AGE

snowflake-3968820950-9dgr8 1/1 Running 0 2m

snowflake-3968820950-vgc4n 1/1 Running 0 2m

And this is great, developers are able to do what they want, and they do not have to worry about affecting content in the production namespace. Let's switch to the production namespace and show how resources in one namespace are hidden from the other. `kubect! config use-context prod`

The production namespace should be empty, and the following commands should return nothing. `kubect! get deployment`

`kubect! get pods`

Production likes to run cattle, so let's create some cattle pods. `kubect! create deployment cattle`

`--image=registry.k8s.io/serve_hostname --replicas=5`

`kubect! get deployment`

NAME READY UP-TO-DATE AVAILABLE AGE

cattle 5/5 5 5 10s

`kubect! get pods -l app=cattle`

NAME READY STATUS RESTARTS AGE

cattle-2263376956-41xy6 1/1 Running 0 34s

cattle-2263376956-kw466 1/1 Running 0 34s

cattle-2263376956-n4v97 1/1 Running 0 34s

cattle-2263376956-p5p3i 1/1 Running 0 34s

cattle-2263376956-sxpth 1/1 Running 0 34s

At this point, it should be clear that the resources users create in one namespace are hidden from the other namespace. As the policy support in Kubernetes evolves, we will extend this scenario to show how you can provide different

authorization rules for each namespace. Feedback Was this page helpful? Yes

No Thanks for the feedback. If you have a specific, answerable question about how to use Kubernetes, ask it on Stack Overflow.

Open an issue in the GitHub Repository if you want to report a problem

or

suggest an improvement. Last modified October 19, 2024 at 5:08 PM PST: Move 'Namespaces Walkthrough' to tutorials section (af9c4c83f4)